catanrl

Reinforcement learning for two-player Catan: research report

Living document, updated with each experiment · last updated 9 October 2026 · github.com/mcrco/catanrl

1 Summary

catanrl trains agents for two-player Catan under Catanatron's rules [1] (15 victory points to win) and measures them against F, Catanatron's `ValueFunctionPlayer`. The current best agent is **graph-s**, a 0.5M-parameter graph network ported from Canopy's Nexus-v3 network [2]. It is trained by imitation of F (DAgger), then self-play PPO on cppanatron, a native C++ port of Catanatron. Its raw policy beats F in 89.6% of games. With 256 simulations of MCTS that does not see the hidden cards it reaches 90.5%, and with 1024 simulations of search on the true game it reaches 92.7%.

Table 1: Headline results. "Fair" agents use only information a real player has. All rows are seat-balanced; the evaluation protocol is in Section 3.

Agent (graph-s reference, update 1500)	Hidden cards	Games	Win vs F, 95% CI
Raw policy, native engine	sees them	10,000	**89.58** 88.97–90.16
Raw policy, belief-averaged	fair	2,000	**89.65** 88.24–90.91
MCTS 256, determinized 1 × 256	fair	1,000	**90.5** 88.5–92.2
MCTS 256 on the true game	sees them	1,000	91.3 89.4–92.9
MCTS 1024 on the true game	sees them	1,000	**92.7** 90.9–94.2

Key findings.

1. **Architecture.** On a teacher-neutral outcome-prediction test the graph network beats an MLP, which beats the cross-dimensional CNN (outcome loss 0.396 vs 0.464 vs 0.83 nats), with 30× fewer parameters than the CNN (Section 4.4).
2. **Recipe.** DAgger from F, then MARL PPO with $\gamma = 0.99$, then a $\gamma = 1$ continuation produces the reference. The $\gamma = 1$ stage adds 1.6 points over a matched $\gamma = 0.99$ control (Section 5.3).
3. **Warm start.** From-scratch runs with $\gamma = 1$ and a KL-adaptive learning rate reach 86–87% vs F after 94–98M environment steps but lose head-to-head to the reference (45.3–48.1%).
4. **Size.** graph-s is large enough. A graph-m replica of the reference lineage ties it head-to-head (50.6%) at about 2.3× the training cost per step.
5. **Reward.** A pure win/loss (WDL) reward trains MARL PPO from scratch at least as well as the shaped reward (51.9% head-to-head, $p \approx 0.02$), and its critic is already zero-sum, as search needs.
6. **Search.** Native MCTS improves every model, and the gain grows with budget: +0.7, +2.1 and +3.5 points at 64, 256 and 1024 simulations for the reference, and +5.0 points at 256 for the WDL model.
7. **Hidden information is worth under a point.** Belief averaging matches the oracle for raw policies, and determinized MCTS is within 0.4–1.0 points of oracle MCTS (not significant). One sampled hand works as well as eight. Going forward, all evaluations use determinizations.
8. **Measurement.** Selecting checkpoints on training-time evaluations is optimistic by up to 2 points. Win rate vs F and head-to-head can disagree. Every model wins more from the first seat, so all evaluations are seat-balanced.

Contents

1 Summary 1
2 Problem and setup 2
3 Evaluation protocol 3
4 Architectures 3
5 Training 5
6 Results 7
7 Search at deployment 8
8 Ongoing 9
9 Earlier experiments 9
10 Credits and third-party sources 11
References 11
A Experiment registry 13
B Updating this report 13

2 Problem and setup

2.1 Game rules

Games use our fork of Catanatron [1] (submodule `catanatron/`) with the BASE map, two players, **15 victory points to win**, a **discard limit of 9** (a rolled 7 makes any player holding more than 9 cards discard half), and **official-spiral number placement** (terrain and ports are shuffled; number tokens follow Catan's official spiral). Games are truncated at **1000 turns**, which counts as a draw. The search matchups recorded no draws.

2.2 Action space

Every decision is one index into a fixed table of **313 actions** for two-player BASE (Table 2). Illegal actions are masked out of the policy logits. Player-to-player trades are not in the table, so agents trade only with the bank and ports.

Table 2: The 313-action flat action space for two-player BASE.

Action type	Count	Notes
Build road / settlement / city	72 / 54 / 54	one per edge or node
Move robber	38	19 tiles × (steal from the opponent, or no steal)
Maritime trade	60	give/receive resource pairs at 4:1, 3:1 and 2:1
Play Year of Plenty / Monopoly	20 / 5	resource pairs (with repeats) / resources
Discard	5	one card at a time
Roll, end turn, buy development card, play knight, play road building	5	one each

2.3 Observations and hidden information

An observation is a numeric vector plus a board tensor of 16 channels on Catanatron's 11 × 21 lattice (3,696 values). Three information levels are implemented (Table 3).

Table 3: Observation levels. All current models use `full` for both actor and critic.

Level	Numeric features include	Numeric	Total
`private`	the board, public state and the player's own hand	58	3,754
`public`	plus the opponent's resource counts (1v1 only)	63	3,759
`full`	plus the opponent's development cards (in hand and playable), its actual VP including hidden VP cards, and whether it has played a card this turn	74	3,770

In 1v1 every resource movement is public (each robber steal is seen by both players), so resource hands can be reconstructed from history. The only hidden information is the opponent's **development-card hand**. Its belief can be enumerated exactly: a multivariate hypergeometric over the unseen development-card pool, excluding hands whose VP cards would already have won. This gives two fair ways to deploy a network trained on `full` observations: average its policy over the belief, or search over determinizations (Section 3.3).

2.4 Engine: cppanatron

cppanatron [3] is a C++20 port of Catanatron (GPL-3.0-or-later, submodule `cppanatron/`), exposed to Python through a nanobind package. It implements the full rules for MINI, BASE and TOURNAMENT maps with 2–4 players, the flat action table, the pinned F heuristic, and a pull-based PUCT search (`MCTSSearch`) with exact chance outcomes. A differential test suite replays random games against Catanatron in lockstep and requires identical state snapshots and legal-action sets before every transition. Batched games (`GameBatch`) write observations straight into caller-owned NumPy buffers. On top of it catanrl builds a vectorized PufferLib [4] multi-agent environment (`--env-backend cppanatron-native`), native evaluations against F, and native head-to-head play.

2.5 The opponent F

F is Catanatron's `ValueFunctionPlayer`. It is a one-ply greedy player: it applies each legal action to a copy of the game, scores the result with a hand-tuned value function, and takes the best. It is fast and strong enough to separate our models. The native port in cppanatron is tested for parity with the Python original.

3 Evaluation protocol

3.1 Standard evaluations

Table 4 lists the evaluations used in this report. All of them play both seats equally and use fixed seeds. Held-out results use seed 67. Training-time evaluations, which select checkpoints, use seed 123, so selection never sees the held-out games.

Table 4: Standard evaluations. "Reference" is the graph-s run `m-graph-s-full-full-native-g1-from-warm-s42-20261001` at update 1500.

Evaluation	Engine	Games	Policy and purpose
vs F (native)	cppanatron	5,000 / seat	greedy, full-information actor; the main strength number
belief-averaged vs F	Catanatron	1,000 / seat	greedy over the belief-averaged policy, plus the oracle on the same seeds; the fairness check
H2H vs reference	cppanatron	2,000 / seat	both greedy; strength relative to the fixed graph-s reference
search vs F	cppanatron	500 / seat	raw or MCTS agents on paired seeds (`scripts/eval_search_matchup.py`)
training-time	cppanatron	2,000 + 500	every 250 updates: vs F and H2H vs the reference, seed 123

3.2 Statistics

Win rates carry Wilson 95% intervals [5]. Half-widths are about ±0.6 points at 10,000 games near 88%, ±1.5 at 2,000 games and ±2 at 1,000. When two agents play the same seeds (search matchups, belief-averaged evaluations), differences are tested with an exact two-sided McNemar test on the discordant games [6]. Head-to-head results use an exact binomial test against 50%. Pairing removes board-to-board variance, but many games are still discordant (for example 171 vs 136 of 2,000 between the $\gamma = 1$ and $\gamma = 0.99$ continuations of Section 5.3), because small policy changes send games down different paths.

3.3 Fairness to hidden information

The native vs-F evaluation feeds the full-information actor the true state, so it sees the opponent's development cards. Two fair counterparts check that this does not inflate results:

- **Belief-averaged raw policy.** The agent plays $\operatorname{argmax}_a \sum_h P(h)\pi(a \mid s_h)$, where s_h is the state with the opponent's hand replaced by hypothesis h from the exact belief. Across 20 belief/oracle evaluations the oracle and the belief-averaged policy differed by at most 1.1 points, and usually by 0.6 or less. For the reference: 90.05% oracle vs 89.65% belief-averaged on the same 2,000 games.
- **Determinized search** (`mcts:sims=N,dets=K`) draws K hands from the exact belief, searches each determinized game with a share of the simulations, and sums root visits (Section 7.3).

3.4 Selection bias, determinism and seats

- **Winner's curse.** "Best" checkpoints are chosen by the maximum of noisy training-time evaluations, so their training-time score is optimistic. With 400-game evaluations, the $\gamma = 0.99$ graph-s run selected update 3000 at 89.0%; on held-out games it scored 87.3% (native, 10,000 games) and 86.95% (belief-averaged). With 2,000-game evaluations the bias shrank below a point (graph-m from scratch: 85.7% at selection, 85.0% held out), and that run's final checkpoint beat its "best" on held-out games (86.1% vs 85.0%).
- **Determinism.** Native evaluation is deterministic: greedy actions on seeded games. Re-running a checkpoint reproduces its result exactly. For example, one graph-m checkpoint evaluated under two names (`best` and update 2966) scored an identical 82.43%.
- **Seats.** Every model wins more often from the first seat, by 2 to 4 points over 5,000 games per seat (reference: 91.2% first, 88.0% second). Sampling noise on that difference is about ±1.3 points, and the gap appears for every model, so it is structural: the first player has an advantage. No result here uses a single seat.
- **vs F is not head-to-head.** The graph-m warm replica scores 2.3 points below the reference against F but ties it head-to-head (50.6%). We report both.

4 Architectures

All current models use **separate** policy and critic networks with the same architecture and `full` observations ("full/full separate"). Earlier shared policy–value networks collapsed under PPO (Section 9).

4.1 Cross-dimensional CNN (xdim)

The xdim backbone follows Gendre and Kaneko's cross-dimensional network for Catan [7]. The board tensor goes through three convolutions (3 × 5 kernels, batch norm, ReLU), is flattened and projected. The numeric features go through an MLP, and a two-layer MLP fuses both branches. The D-M size used since Phase 1 (channels 64/128/128, widths 2048) has about **78.5M parameters per network**, 60.6M of them in the projection of the convolution output. D-S (32/64/64, width 1024) has 19.8M.

4.2 MLP

A plain MLP over the concatenated observation, used as a baseline in the architecture ablation (`mlp-512`: 2.4M parameters; `mlp-2048`: 12.6M).

4.3 Catan graph network (graph-s, graph-m)

Attribution. The `catan_graph` backbone (`CatanGraphBackbone` in `src/catanrl/models/backbones.py`), its graph policy and value heads (`src/catanrl/models/heads.py`) and the graph-s/graph-m presets are a port of the **Nexus-v3 network from Canopy by cullback** [2] (github.com/cullback/canopy, tag `catan-nexus-v3`, commit `6185983a88ba6802e7fa9893cef5a76a15de2595`; files `examples/catan/model/nexus_v3.rs` and `examples/catan/encoder/nexus_v3.rs`), adapted to Catanatron's observation vector and action space. Canopy has no license file.

The backbone gathers the board's active nodes from Catanatron's lattice, linearly recovers per-tile resource and robber planes from each tile's six-node incidence pattern, and adds normalized and semantic inputs. Pre-norm residual layers then pass messages node↔node and tile↔node. The policy head is structured: per-node logits for settlements and cities, per-edge logits for roads, per-tile logits for the robber, and the remaining actions from pooled features. The value head is a two-hidden-layer MLP over pooled features, either scalar (PPO) or win/draw/loss.

Table 5: Network sizes (configs `catan-graph-{s,m}-flat-2p-full-sep.yaml`, `xdim-flat-2p-full-sep.yaml`).

Preset	Hidden	Global	Layers	Head	Parameters (policy + critic)
graph-s	128	64	3	128	0.48M + 0.37M
graph-m	256	96	4	256	2.16M + 1.74M
xdim D-M	—	—	3 conv	2048	≈78.5M + ≈77.9M

4.4 Architecture ablation

The ablation on branch `arch-ablation` (`scripts/arch_ablation/FINDINGS.md`) trained 15 candidates by supervised learning (AdamW, cosine schedule, batch 512, 6,000 steps, best-validation checkpoint) on 2-player BASE positions with full observations. Two teachers generated the data with native search: a graph network and a CNN. The only target no network can bias is the **terminal game outcome** on held-out games pooled from both teachers. Table 6 reports that loss (lower is better; 0.693 is a coin flip).

Table 6: Teacher-neutral outcome loss (win/draw/loss cross-entropy, nats) on held-out games. Graph < MLP < CNN.

Family	Best candidate	Params	Pooled	Graph teacher	CNN teacher
graph, graph transformer	`graph-s`, `gtx-s`	0.5M / 0.9M	**0.396 / 0.397**	0.48 / 0.49	0.31 / 0.30
graph (larger)	`graph-m`	2.4M	0.434	0.53	0.33
MLP	`mlp-512`	2.4M	0.464	0.60	0.35
MLP (larger)	`mlp-2048`	12.6M	0.540	0.67	0.41
CNN (xdim)	`xdim-default`	16.8M	0.831	1.13	0.53
CNN (xdim)	`xdim-compact-m`, `xdim-d-s`	2.0M / 19.8M	0.91 / 0.88	1.29 / 1.16	0.53 / 0.59

Findings:

- **Graph < MLP < CNN.** The graph networks' outcome loss is about half the CNN's and 15% below the MLP's, with 4–30× fewer parameters. The CNN predicts the winner **worse than a coin flip** at every size from 2M to 20M parameters. Its validation loss is best after 250–750 steps and then rises: the value head memorizes. The same order held on search-value targets, with seed spread of ±0.001–0.007.
- **Policy imitation is biased toward the teacher's family.** Search targets carry the teacher's logit shape, so each family fits its own teacher's policy best. Imitating F's argmax (the old DAgger proxy) has the same flaw. Do not compare architectures by imitation loss.

- **Within the graph family.** A transformer over the same node and tile tokens matches message passing, so the structured head is what matters. Semantic inputs help the value (removing them costs about 10σ). Size does not matter at this data scale (about 110k samples; every model overfits within 1–2 epochs): graph-s (0.5M) is at least as good as graph-m (2.4M) and graph-l (6.8M).
- **Speed.** Training step at batch 512: MLP 3 ms, graph-s 26 ms, xdim D-M 120 ms. The graph nets are launch-bound (about 200 small kernels per forward). CUDA graphs (`torch.compile(mode="reduce-overhead")`) cut graph-s from 3.0 ms to 0.39 ms at batch 1.

Supervised fit is only a proxy. The decisive test is RL strength under the same recipe, which the CNN replica now running tests (Section 8).

5 Training

5.1 DAgger from F

Every warm-started lineage begins with DAgger [8] against F on the native engine: 24 iterations of 4,096 steps on 8 environments, executing the expert's action with probability β (1.0, multiplied by 0.9 per iteration, floor 0.1), a dataset capped at 800k samples, one epoch per iteration (batch 512, learning rate 3e-4). A critic is fitted to the collected shaped returns at the same time. graph-s reached 47.5% vs F (200 games) after 24 iterations.

5.2 Self-play MARL PPO with a central critic

`train-marl-cc` trains one policy that plays both seats against itself on the native vectorized environment. Each environment step stores a row for each player. PPO [9] uses the clipped objective with GAE [10] and a centralized critic in the style of MAPPO [11]. The trainer supports asymmetric actor and critic observations (for example a `public` actor with a `full` critic). Current runs use full/full separate networks, deployed through belief averaging or determinized search. Table 7 lists the two recipes in use.

Table 7: MARL PPO settings. Launch scripts set every flag explicitly; see Appendix A for experiment names.

Setting	Warm-start lineage (reference)	From-scratch recipe
Environments × rollout steps	128 × 128 = 16,384 environment steps per update	
Epochs, minibatch	2 epochs, 2,048 rows	
Policy LR	5e-5, constant	2e-4 start, KL-adaptive in [1e-5, 3e-4], target KL 0.01
Critic LR	2e-4	
γ / GAE λ	0.99 for 3,000 updates, then 1.0 for 1,500 / 0.95	1.0 / 0.95
Clip ε, value coef, grad clip	0.2, 0.5, 0.5	
Entropy coefficient	0.001	0.003 ⟶ 0.001, linear over the run
KL early stop (`--target-kl`)	0.02 (stop at 0.03)	0.05 (stop at 0.075)
Reward	shaped	shaped, or WDL
Budget	3,000 + 1,500 updates (73.7M steps)	8.75 h cap: 94–98M steps for graph-s

5.3 Horizon: $\gamma = 1$

Training games last about 250–340 environment steps. With $\gamma = 0.99$ the credit for the first move decays to $0.99^{290} \approx 0.05$ over a game of about 290 steps. The opening placements largely set a player's production for the whole game, so they see a reward about 20 times smaller than it is. With $\gamma = 1$ the critic predicts the undiscounted outcome, and GAE's $\lambda = 0.95$ still controls variance.

The test was a matched pair of continuations from the same checkpoint (update 3000 of the $\gamma = 0.99$ run), 1,500 updates each:

Table 8: $\gamma = 1$ vs a matched $\gamma = 0.99$ control. Difference: +1.6 points native ($z \approx 3.5$), +1.75 points belief-averaged (paired McNemar $p = 0.05$).

Continuation	vs F native (10,000)	1st / 2nd seat	Belief-averaged (2,000)
$\gamma = 1$ (the reference)	**89.58** 88.97–90.16	91.2 / 88.0	**89.65** 88.24–90.91
$\gamma = 0.99$ control	88.01 87.36–88.63	89.2 / 86.8	87.90 86.40–89.26
source (update 3000)	87.31 86.64–87.95	88.9 / 85.8	86.95 85.40–88.36

Training-time evaluations of both continuations stayed between 85.6% and 88.8% (2,000 games each). The gain was visible only in the large held-out evaluations.

5.4 Near-zero value-head initialization

With $\gamma = 1$, bootstrapping never removes a constant offset in the critic. The graph value head's output layer reads non-negative ReLU features, and with gain-1 orthogonal initialization a fresh critic predicted an arbitrary per-seed constant between about −2 and +5. In the first from-scratch graph-m run the initial value was about +5.2, so every game-ending transition looked worse than expected. The policy learned to stop building and stall every game to the turn limit, and the run was aborted at 4.5M steps. The output layer now uses gain 0.01, so fresh critics start within ±0.05 of zero (commit `c5d653e`). The graph-m runs and the WDL run use it; graph-s scratch v2 predates it.

5.5 KL-adaptive policy learning rate

After every PPO update the policy learning rate is divided by 1.5 if the update's mean KL exceeds twice the target, multiplied by 1.5 if it is below half the target, and clamped to [1e-5, 3e-4] (`--policy-lr-schedule kl-adaptive`). This is the `AdaptiveScheduler` of rl_games by Denys Makoviichuk [12] (the `schedule: adaptive` option in IsaacGymEnvs' PPO configs), which applies the adaptive KL penalty rule of the PPO paper [9] (Sec. 4) to the step size. A constant rate failed both ways. At 2e-4 the first from-scratch graph-s run hit the KL early stop on about 99% of updates, and at 5e-5 progress is slow once the policy sharpens. The adapted rate settled at about 0.9–1.3e-4 for graph-s and 4–6e-5 for graph-m.

The early stop must be loose enough not to hide the KL from the schedule. KL grows across an update's minibatches, so the mean over the minibatches that ran sits near half the stop threshold. With `--target-kl 0.02` (stop at 0.03), about 97% of updates stopped early, their mean KL (about 0.011) stayed inside the adaptive band, and the rate sat at its cap. That run (scratch v1) was aborted. v2 uses `--target-kl 0.05`, and the trainer now warns when 1.5 × `target_kl` is below four times the adaptive target.

5.6 Rewards

- **Shaped** (`--reward-function shaped`): +1 on the winning transition, otherwise $0.01 \cdot \Delta\mathrm{VP}\ /15 + 0.0025 \cdot \Delta\text{production}$, where production is the player's own expected pips. A loss pays nothing, so the critic estimates the win probability plus a little shaping.
- **WDL** (`--reward-function win`): +1 for a win, −1 for a loss, 0 for a draw or truncation, with no shaping. The critic is zero-sum.

Search negates values across players, so it needs zero-sum values. A shaped critic's output p must be mapped to $2p - 1$ (Section 7).

5.7 Throughput

One RTX 2070 SUPER (8 GB) and a 31 GB host run everything. Table 9 lists the changes that made 100M-step runs fit in a working day.

Table 9: Throughput work. A full graph-s run sustains about 3.1k environment steps/s including evaluations (98.3M steps in 8.75 h); graph-m about 1.3k.

Change	Effect
Native vectorized MARL environment on cppanatron	replaces Python Catanatron rollouts and evaluations
Policy forward only on decision rows	the idle seat and forced moves have one legal action; in earlier runs only ≈29% of agent rows were real decisions
Structured-dtype batch decode	5.3 ms → 0.13 ms per 256 full-observation rows, byte-identical
GPU rollout buffer (`--rollout-buffer-device auto`)	minibatches gathered on the GPU; full/full actor and critic share one stored state; 0.47 GiB for graph-s beside a 1.81 GiB training peak
On-device critic values during collection	the environment step overlaps the critic forward
All of the above	collection 6.3k → 10.6k environment steps/s; end to end 2.65k → 3.74k steps/s (first 50 updates)
Native evaluations vs F and H2H	5–7× faster than the Python evaluator, matching within noise (89.0% vs 87.5% on 400 games)
W&B action-type chart rebuilt only at eval cadence	it had grown to about 2 s per update, about an hour per 8-hour run

6 Results

6.1 Strength against F and the reference

Table 10: Main results (win %, Wilson 95% CI). "Steps" counts MARL environment steps; warm-started lineages also had DAgger. From-scratch runs use $\gamma = 1$, the KL-adaptive LR and entropy decay. H2H is the row's model vs the reference, 2,000 games per seat.

Model	Steps	vs F native (10,000)	1st / 2nd	Belief-avg (2,000)	H2H vs ref. (4,000)
graph-s reference (warm start + $\gamma = 1$, update 1500)	73.7M	**89.58** 88.97–90.16	91.2 / 88.0	**89.65**	—
graph-m warm replica (same lineage, only the network differs)	73.7M	87.33 86.66–87.97	88.6 / 86.1	87.50	50.6 49.1–52.2
graph-s scratch v2 (shaped), best (update 5750)	98.3M	86.74 86.06–87.39	88.4 / 85.1	86.85	45.5 44.0–47.0
final (update 5999)		86.05 85.36–86.72	87.6 / 84.5	—	45.5 43.9–47.0
graph-m scratch (shaped), final (update 5999)	98.3M	86.08 85.39–86.74	87.5 / 84.6	—	48.1 46.5–49.6
best		84.96 84.25–85.65	86.0 / 84.0	83.95	45.6 44.1–47.1
WDL graph-s scratch, best (update 5000)	94.4M	86.54 85.86–87.19	87.5 / 85.6	87.35	45.3 43.8–46.9
final (update 5761, 8.75 h cap)		85.01 84.30–85.70	86.8 / 83.2	—	46.0 44.5–47.5

Table 11: Pairwise native head-to-head results between candidate models.

Head-to-head (A vs B, 4,000 games)	A wins	95% CI	p vs 50%
graph-m scratch final vs graph-s scratch v2 final (equal steps, 98.3M)	**53.3**	51.8–54.9	3e-5
graph-m scratch vs graph-s scratch v2 at about 49M steps	52.6	51.0–54.1	0.001
graph-m warm replica vs graph-m scratch final	51.9	50.3–53.4	0.02
graph-m warm replica vs graph-s reference	50.6	49.1–52.2	0.46
WDL best vs shaped v2 final	**51.9**	50.4–53.4	0.02
WDL final vs shaped v2 final	51.9	50.3–53.4	0.02

6.2 Warm start beats training from scratch

The warm-started reference used 73.7M MARL steps after a short DAgger stage. Every from-scratch run, at 94–98M steps, is about 3–4.5 points weaker against F and loses head-to-head to it (45.3–48.1%). Imitating F gives a head start that self-play alone did not make up within these budgets. The from-scratch curves were still rising slowly when their time caps ended (Figure 1), so longer runs might close part of the gap.

6.3 graph-s is large enough

The graph-m warm replica, identical to the reference lineage except for the network, ties the reference head-to-head (50.6%, $p = 0.46$) and is 2.3 points **lower** against F. From scratch, graph-m edges graph-s at equal steps (53.3% head-to-head at 98.3M steps, 52.6% at 49M), but it takes about 2.3 times longer per step (about 20 h vs 8.75 h for 98.3M steps). The warm-started graph-m beats scratch graph-m (51.9%), as for graph-s. We keep graph-s.

6.4 Win/loss reward trains from scratch at least as well as shaping

The WDL run uses the shaped v2 recipe with a different reward. The only other difference is the near-zero value-head initialization, which v2 predates. WDL led clearly at the first evaluation (41.5% vs 28.7% vs F at 4.1M steps). The curves were level through about 45M steps, and WDL was slightly ahead on most evaluations after about 49M (Figure 1). In direct play WDL beat shaped v2's final checkpoint 51.9% over 4,000 games (±1.5 points, $p \approx 0.02$), even though WDL stopped at 94.4M steps against v2's 98.3M. Its belief-averaged score (87.35%) is the highest of the from-scratch runs. WDL's critic is zero-sum by construction, which suits search directly, and it gains the most from MCTS (Section 7). Shaped reward has no demonstrated advantage for from-scratch MARL PPO here.

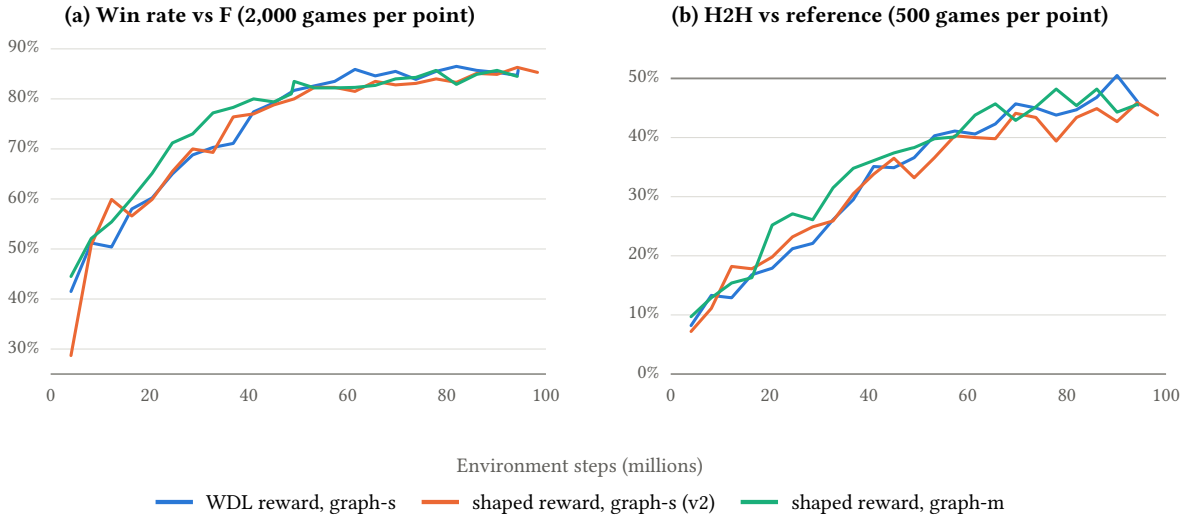


Figure 1: From-scratch training curves (training-time evaluations: native engine, seed 123, greedy full-information actor). Data: `report/data/scratch-curves.json`, read from the local W&B run files.

7 Search at deployment

7.1 Native MCTS

Search uses cppanatron's `MCTSSearch` [3], a PUCT search in the style of AlphaZero [13]. Priors come from the policy network and leaf values from the critic. Chance nodes are exact (dice, development-card draws, robber steals), backup is perspective-aware, and the move is the argmax of root visits ($c_{\text{puct}} = 1.5$, no Dirichlet noise). The search is pull-based: the caller evaluates the root and each selected leaf with its own model, so Python can batch inference across many searches. Matchups run through `scripts/eval_search_matchup.py` on 500 paired seeds per seat (seed 67). Deltas in Table 12 are exact McNemar tests against the same model's raw policy on the same games.

Table 12: Native MCTS vs F (win %, 1,000 paired games each; CIs about ±2 points). These searches run on the true game and see the opponent's hidden cards; Table 13 gives the fair version.

Model	Raw	MCTS 64	MCTS 256	MCTS 1024
graph-s reference (shaped)	89.2	89.9 (+0.7, $p = 0.37$)	91.3 (+2.1, $p = 0.02$)	**92.7** (+3.5, $p = 0.0002$)
graph-s scratch v2, final (shaped)	87.6	89.3 (+1.7, $p = 0.03$)	91.1 (+3.5, $p = 0.0003$)	—
graph-s WDL scratch, best	85.7	88.4 (+2.7, $p = 0.002$)	90.7 (+5.0, $p = 5 \times 10^{-6}$)	—
Network evaluations per move	1	63	249	983
Wall clock per 1,000 games	13–16 s	8–11 min	40–45 min	6.8 h

Search helps every model, and the gain grows with budget. The WDL model, with the weakest raw policy, gains the most and reaches the same 256-simulation strength as the shaped models (90.7% vs 91.1–91.3%). Its critic is trained on the zero-sum target that the search backs up.

7.2 Values must be zero-sum

The search negates a child's value when the player to move changes, so it assumes $V_{\text{me}} = -V_{\text{opponent}}$. A shaped critic estimates a win probability p (a loss pays 0) plus a little shaping, so its values must be mapped to $2p - 1$ before they enter the tree. The search matchup harness does this (value transform `win-probability`); WDL critics pass through unchanged (`zero-sum`). The older native-MCTS scripts (`scripts/eval_native_mcts_*.py`) fed raw shaped values. Raw values in $[0, 1]$ are non-negative for whoever is to move, so after negation every position with the opponent to move looks bad for us, and those searches avoid moves that end the turn. Their results are not comparable with Table 12.

7.3 Hidden information: determinized search

`mcts:sims=N,dets=K` makes MCTS fair. At each decision it draws K hands from the exact 1v1 belief over the opponent's development cards. For each distinct hand, `Game.determinize_development_cards` sets that hand and rebuilds the deck so that draws inside the tree carry no trace of the true hand. Each determinized game gets a

proportional share of the N simulations, root visits are summed, and the agent plays the argmax. This is perfect-information Monte Carlo at the root [14], a cheap relative of information-set MCTS [15].

Table 13: Determinized vs oracle MCTS at 256 total simulations (win % vs F, the same 1,000 games). Deltas in the determinized columns are against the oracle. For the reference, 1 × 256 vs 8 × 32: +0.2 points, $p = 0.83$.

Model	Raw	Oracle 256	Determinized 8 × 32	Determinized 1 × 256	Det. 8 × 32 vs raw
graph-s reference	89.2	91.3	90.3 (-1.0, $p = 0.18$)	90.5 (-0.8, $p = 0.30$)	+1.1 ($p = 0.27$)
graph-s WDL scratch, best	85.7	90.7	90.3 (-0.4, $p = 0.67$)	—	**+4.6** ($p = 2 \times 10^{-5}$)

Knowing the hidden cards is worth under a point and is not significant on its own. One sampled hand does as well as eight. Determinized MCTS still beats the raw policy clearly for the WDL model, but only marginally for the reference at this budget. **Going forward, all evaluations use determinizations.**

7.4 Throughput lessons

- **Kernel launches dominate small batches.** The graph backbone's forward costs about 6 ms whatever the batch, up to about 32 rows. Replaying CUDA graphs captured at power-of-two batch sizes cuts it to about 2 ms (`catanrl.eval.batched_inference`).
- **One pending leaf per search.** The native search keeps one pending leaf, so per-game workers made every simulation a GPU round trip (about 2.4k evaluations/s). Lockstep batching advances many boards in one process to their next network request and evaluates them in one batch. It reaches about 10k evaluations/s and plays identical games (`catanrl.eval.batched_native_mcts`).
- **Memory.** Native expansion copies a game per child: about 300 µs per simulation, about 39 MB per 256-simulation tree and about 117 MB per 1024-simulation tree. Size concurrency so that processes × boards × tree size stays well under RAM. One oversized run exhausted the 31 GB machine, and the 1024-simulation run used 12 boards per process instead of 32.

A comparison of 2FFS with native MCTS lives on branch `twoffs`.

8 Ongoing

CNN replica of the reference lineage (running). The architecture ablation ranked the CNN last on a supervised proxy. This run tests whether a CNN can match the graph network under the identical RL recipe. It replicates the graph-s reference lineage with the xdim D-M CNN (full/full separate, about 78.5M parameters per network): DAgger (`dagger-xdim-full-native-s42-20261009`), MARL $\gamma = 0.99$ for 3,000 updates (`m-xdim-full-full-native-warm-s42-20261009`), then $\gamma = 1$ for 1,500 updates from update 3000 (`m-xdim-full-full-native-g1-from-warm-s42-20261009`). Every setting matches the graph-s and graph-m lineages; training-time H2H is against the graph-s reference. The post-evaluation will run native vs F (5,000 games per seat) and native H2H vs the reference (2,000 per seat) for update 1500, `best` and the latest checkpoint, plus a belief-averaged evaluation of update 1500. **Results pending.**

Open questions.

- Does a WDL warm-start lineage (DAgger, then WDL MARL with $\gamma = 1$) beat the shaped reference, given that WDL from scratch beat shaped from scratch?
- Can search improve training, not just play? Frozen-teacher distillation with the old CNN gave at most about one point (Section 9). The graph network, native search and zero-sum critics change the setup enough to revisit it.

9 Earlier experiments

These experiments came before the native engine, the graph network and the current evaluation protocol. They used the xdim CNN, mostly the Python Catanatron engine, 500–2,000-game evaluations and board generation from before official-spiral number placement became the default. Their absolute win rates are **not comparable** with the sections above; the findings carry over. Launch commands are in the scripts named below and in git history (`README.md` before this report).

9.1 Phase 1: DAgger model-size sweep (xdim)

DAgger from F (40 iterations, 8,192 steps per iteration, β decay 0.97 to 0.1) over xdim sizes D-S (1024 × 2), D-M (2048 × 2), D-L (3072 × 2) and D-XL. D-M learned faster and better than D-S. D-L gave no clear gain for its width, so D-XL was skipped. A shared policy–value backbone matched separate networks under DAgger. D-M became the default size until the graph network replaced it.

9.2 Phase 2: SARL and MARL, shared vs separate, full vs public

- **SARL PPO vs F** (`train-sarl-ppo`, 16.4M steps, warm-started from DAgger). The separate public-actor/full-critic run reached 57.2% / 56.8% (first / second seat). The **shared** public policy–value network improved early and then degraded badly in three runs (baseline, value coefficient 0.25, policy LR 5e-5 with value coefficient 0.25). A control that split the shared checkpoint into separate public policy and public critic networks did not collapse and became the strongest SARL policy, which points at the shared trunk rather than the public critic.
- **MARL self-play with a central critic.** `m-sep-pub-full` (public actor, full critic) beat the first SARL policy 61.2% and the stronger separate public/public SARL policy 52.1% over 2,000 games (95% CI 49.9–54.3%).

9.3 Phase 2.5: MCTS and deployment ablations (Python IS-MCTS)

With 8 determinizations × 64 simulations, search lifted the shaped MARL policy from 52.05% to 56.75% vs F (+4.70, McNemar $p = 0.0008$, 2,000 games). An earlier 56.55% result was invalid: it paired the policy with a critic from a different update, so policies and critics are now saved as pairs. A win/loss-reward variant ($\gamma = 0.999$, $\lambda = 0.99$) had a much weaker raw policy (44.85%) and a +2.45-point search gain that was not conclusive ($p = 0.085$). Modeling the opponent inside search as F rather than as self-play did not help (54.25% vs 56.75%, $p = 0.079$). The determinization-vs-depth sweep was not completed.

9.4 Phase 2.6: PPO update intensity

An 8.2M-step screen of epochs, batch size and learning rate (seed-123 selection). Only halving the policy LR (T3: 2 epochs, batch 1,024, policy LR 5e-5) improved steadily: best / final / last-3 mean 56.2% / 55.4% / 54.7%, against 47.2% / 43.0% / 43.8% for the old setting. More minibatches at the old LR (T2) peaked early and degraded, so the actor step size was the issue. Two 16.4M-step T3 seeds confirmed a 54–55% plateau.

9.5 Phase 2.7: full-information actor and belief averaging

A full/full xdim model (DAgger, then MARL) scored **58.50% as the oracle and 58.50% belief-averaged**, against 52.05% for the public-actor reference. Training on full information and marginalizing over the exact belief at test time lost nothing. Every later model uses this recipe. Python IS-MCTS at 8 × 152 reached 60.70% (+2.2, not conclusive) and took 16.6 hours.

9.6 Phase 3: search distillation (Expert Iteration gate)

`train-alphazero --mode distill` trained the student policy on frozen-teacher MCTS visit targets [16].

- The first run collapsed (57.8% ⟶ 16.0%) because the student loss used an unmasked softmax over all 313 actions. PPO leaves illegal logits unconstrained, so the denominator was meaningless.
- With the mask fixed, targets were almost one-hot (0.022 nats). Raw visit targets (`--target-temperature 1.0`) raised target entropy to 0.22 nats.
- The full 512-game gate gave a student at 59.10% vs the source's 57.80% (+1.30, paired $p = 0.25$) and 50.6% head-to-head ($p = 0.61$).

Expert Iteration was not started: the gain, if real, was about a point.

9.7 Phase 4: rollout length and credit horizon

From the full/full xdim checkpoint, 8.2M-step runs compared 512- vs 2,048-step rollouts and γ/λ = 0.99/0.95 (effective TD horizon about 17 steps) vs 0.999/0.99 (about 91 steps). No variant beat the source (59.8%): R512-H91 reached 59.6% / 56.2% (best / last-3 mean), R2048-H91 58.4% / 54.9%, and R2048-H17 59.4% / 56.5%. Every update hit the per-minibatch KL stop, so the long rollouts used only about 20% of their planned minibatches, which confounds the comparison. The H91 critic was also much harder to fit (explained variance 0.28–0.39 vs about 0.6). A fixed-reuse follow-up (one epoch, no KL stop) beat its source 52.35% head-to-head (2,000 games, $p = 0.04$). Continuing it at half the policy LR won 55.05% ($p = 7 \times 10^{-6}$), even though its small training evaluation peaked

at step 0. Lesson: small training-time evaluations can miss real gains that a held-out head-to-head detects. (These results are from the `stash-marl-h2h-selection` branch's README.)

9.8 Path to the graph-s reference

- `dagger-graph-s-full-native-s42-20260927`: DAgger on the native engine, 47.5% vs F after 24 iterations.
- `m-graph-s-full-full-native-s42-20260927`: MARL $\gamma = 0.99$ for 53.2M steps. Update 3000 was selected at 89.0% on 400 training games and scored 87.3% held out.
- `m-graph-s-full-full-native-fresh-s42-20260928`: graph-s from scratch with $\gamma = 0.99$ and a constant policy LR of 2e-4, which hit the KL early stop on about 99% of updates. It was continued at 5e-5 to 77.8M steps and reached 83.95% belief-averaged (update 4750). This failure motivated the KL-adaptive LR.
- Scratch v1 (KL-adaptive with `--target-kl 0.02`) was aborted at 9.3M steps because the early stop hid the KL. The first graph-m scratch run was aborted at 4.5M steps when it stalled games because of the critic's initial offset.

10 Credits and third-party sources

- **Catanatron** by Bryan Collazo (bcollazo) [1], GPL-3.0: the reference rules engine and the opponent F (`ValueFunctionPlayer`). We use a fork, tracked as the `catanatron/` submodule.
- **cppanatron** [3], GPL-3.0-or-later: our C++20 port of Catanatron, with the native vectorized environment, the native F and native MCTS. Submodule `cppanatron/`.
- **Canopy** by cullback [2], **Nexus-v3 network**, tag `catan-nexus-v3`, commit `6185983a88ba6802e7fa9893cef5a76a15de2595`. Canopy has **no license file**. The `catan_graph` backbone, its graph policy and value heads, and the graph-s/graph-m presets are a port of this network adapted to Catanatron's observation vector and action space. Every result in this report labelled graph-s or graph-m uses it.
- **rl_games** by Denys Makoviichuk [12]: the KL-adaptive policy learning rate is its `AdaptiveScheduler`, which applies the adaptive-KL rule of the PPO paper [9] to the step size.
- **Cross-dimensional network** by Gendre and Kaneko [7]: the basis of the xdim CNN backbone.
- PufferLib [4] provides the vectorized environment interface.

References

[1] B. Collazo, "Catanatron: Settlers of Catan bot simulator and strong AI player." [Online]. Available: https://github.com/bcollazo/catanatron

[2] cullback, "Canopy, Nexus-v3 Catan network." [Online]. Available: https://github.com/cullback/canopy

[3] mcrco, "cppanatron: a C++20 port of the Catanatron engine." [Online]. Available: https://github.com/mcrco/cppanatron

[4] J. Suarez, "PufferLib: Making Reinforcement Learning Libraries and Environments Play Nice." [Online]. Available: https://arxiv.org/abs/2406.12905

[5] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[6] Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[7] Q. Gendre and T. Kaneko, "Playing Catan with Cross-dimensional Neural Network." [Online]. Available: https://arxiv.org/abs/2008.07079

[8] S. Ross, G. Gordon, and D. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 2011, pp. 627–635.

[9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms." [Online]. Available: https://arxiv.org/abs/1707.06347

[10] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," in *International Conference on Learning Representations*, 2016.

[11] C. Yu *et al.*, "The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games," in *Advances in Neural Information Processing Systems*, 2022.

[12] D. Makoviichuk and V. Makoviychuk, "rl-games: A High-performance Framework for Reinforcement Learning." [Online]. Available: https://github.com/Denys88/rl_games

[13] D. Silver *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.

[14] J. Long, N. R. Sturtevant, M. Buro, and T. Furtak, "Understanding the Success of Perfect Information Monte Carlo Sampling in Game Tree Search," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2010.

[15] P. I. Cowling, E. J. Powley, and D. Whitehouse, "Information Set Monte Carlo Tree Search," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 2, pp. 120–143, 2012.

[16] T. Anthony, Z. Tian, and D. Barber, "Thinking Fast and Slow with Deep Learning and Tree Search," in *Advances in Neural Information Processing Systems*, 2017.

A Experiment registry

Runs live under `experiments/<name>/` (gitignored), each with `metadata.json` (exact architecture, game and training config), `checkpoints.json` and checkpoints. `load_experiment(name).build_policy(which=...)` rebuilds any of them.

Table 14: Experiments behind Sections 5–7. Search-matchup outputs: `experiments/twoffs-vs-mcts-20261004`, `wdl-mcts-20261007`, `determinized-mcts-20261008`.

Role	Experiment name
graph-s DAgger	`dagger-graph-s-full-native-s42-20260927`
graph-s MARL $\gamma = 0.99$ (source, update 3000)	`m-graph-s-full-full-native-s42-20260927`
graph-s reference ($\gamma = 1$, update 1500)	`m-graph-s-full-full-native-g1-from-warm-s42-20261001`
$\gamma = 0.99$ control	`m-graph-s-full-full-native-g099-control-from-warm-s42-20261001`
graph-s scratch v2 (shaped)	`m-graph-s-full-full-native-scratch-g1-klada-v2-s42-20261001`
graph-s scratch WDL	`m-graph-s-full-full-native-scratch-g1-wdl-s42-20261007`
graph-m scratch (shaped, zero-init critic)	`m-graph-m-full-full-native-scratch-g1-klada-zinit-s42-20261002`
graph-m DAgger, MARL $\gamma = 0.99$, $\gamma = 1$	`dagger-graph-m-full-native-s42-20261003`
	`m-graph-m-full-full-native-warm-s42-20261003`
	`m-graph-m-full-full-native-g1-from-warm-s42-20261003`
xdim replica (running)	`dagger-xdim-full-native-s42-20261009`
	`m-xdim-full-full-native-warm-s42-20261009`
	`m-xdim-full-full-native-g1-from-warm-s42-20261009`
graph-s from scratch v0 ($\gamma = 0.99$)	`m-graph-s-full-full-native-fresh-s42-20260928`

B Updating this report

- Add new results to the section that answers their question and update Section 1 if a headline changes. Move superseded results to Section 9 with their key numbers.
- Report win rates with Wilson intervals, both seats and the game count. Use paired seeds and McNemar tests when comparing agents on the same games. Evaluate search agents with determinizations.
- Keep plotted data in `report/data/` and compile with `typst compile report/report.typ report/report.pdf`. Commit the PDF with the source.